EE6405 期中考试复习与出题习惯分析

> 课程：EE6405 Natural Language Processing（Dr. S. Supraja 主讲部分）
>
> 考试：**Week 8 · 2026-10-09（周五）· 1 小时 45 分钟 · 现场监考 · 闭卷**
>
> 占比：**35%**
>
> 范围：官方 lesson plan 标注 **Week 1–6**；但 `mid-term-exam/` 归档含 Week 7 截图，**建议按 Week 1–7 准备**（见 §一）
>
> 材料来源：`mid-term-exam/requirement.txt` + `mid-term-exam/week1–week7/` 共 50 张周测截图（去重后 **40 道真题**）+ 各周 `Week*MCQ.md` / notebook / 课件

〇、考场速览：期中规则与周测不一样

这是本册最重要的一条。**周测的负分制在期中被大幅放宽**，策略要跟着改：

项目	周测（Coding / IRA）	
时长	15–20 分钟	**1 小时 45 分钟**
单选题答错	硬错 **−20%**、near miss 0%	**完全不扣分**
多选题答错	错选扣分	扣分（仅理论 IRA-style 多选题），但**总分不为负**
总分下限	可为负	**不为负**
题型	代码单选 / 理论多选	单选 + 多选 + **填空（新）**
公式	—	**不需要背**，计算简单直接

> **三条由此推出的应试结论**：
>
> 1. **单选题绝不空着**——没有惩罚，猜也有分。
> 2. **多选题才需要谨慎**——仍按"不确定的选项不选"处理；但因为总分不为负，比周测安全。
> 3. **别把周测那套"宁可不选"用到单选上**——那是让你白丢分。

一、requirement.txt 逐条解读

#	原文要点	含义与应对
1	Questions with multiple answers will be indicated, else single (one answer)	**看题干是否标明多选**。没标就是单选
2	Penalty is ONLY applicable for theory questions (IRA-style with multiple answers) but **NO negative overall marks**; other types: **no penalty**	扣分只针对"理论型多选题"；**总分不会为负**；其他题型**完全不扣分**
3	Sample under "Supplementary materials" folder，but **not reflective of the difficulty level**	NTULearn 样题只用来熟悉界面，**难度不代表真题**
4	You are **NOT required to memorise any formulas**	**不用背公式**——但要看懂指标含义（F1 / AUC-ROC / BLEU / ROUGE 各衡量什么）
5	填空题：**not case-sensitive**，但**不能拼错**；无英美拼写歧义；**每空一个词或一个字母**（不能多词）	大小写随意，**拼写必须对**
6	可用**一张空白 A4**（must be empty）；**不能**用整本 / foolscap pad；不能用于与同学交流	撕**一张**空白 A4 当草稿纸
7	1h45min；去洗手间要**举手**由监考陪同，**不能带手机**	考前先去洗手间
8	提前交卷可离开，但**必须先向 TA 展示 NTULearn 提交完成**；不能留在教室等朋友	交卷流程别漏
9	**CLOSED-BOOK**，关掉并收好所有设备（phone / smartwatch / AirPods / earbuds）	手表也要摘

关于范围的说明

来源	说法
`Learning_note.md`（课程 lesson plan）	Mid-term 覆盖 **Week 1–6**
`mid-term-exam/` 归档	含 **week1–week7** 全部周测截图

结论：以 Week 1–6 为保底，**Week 7 也要看**（内容量最小）。本册按 **Week 1–7** 覆盖。

二、出题习惯分析（从 40 道真题逆向归纳）

2.1 题型总览

周	截图数	去重后题数	题型	主题
Week 1	10（1–5 与 6–10 **完全重复**）	**5**	单选	Preprocessing（NLTK / 正则）
Week 2	10（两组各 5）	**10**	5 单选 + 5 多选	Linguistic features（spaCy / WSD / NER / POS / DP）
Week 3	5	**5**	单选	TF-IDF 预处理 / PCA 语义 / **SRL**
Week 4	5	**5**	**多选**	Topic modeling / CRF / 评估指标 / 词嵌入 / 分类器
Week 5	5	**5**	单选	Neural models / tuning / GNN 代码
Week 6	5	**5**	**多选**	Transformer / regularisation / ViT
Week 7	5	**5**	单选	Attention 代码 / LayerNorm / ToT-GoT
合计	50	**40**	—	—

> ⚠️ **归档提示**：`mid-term-exam/week1/6.png–10.png` 是 `1.png–5.png` 的**完全重复**截图（同题、同选项、同得分，仅滚动位置不同）。`week2/` 的 10 张则是**两组不同的 5 题**，不作重复处理。

题型规律：代码题 = 单选，理论概念题 = 多选。这条几乎贯穿全部 7 周。

2.2 ⭐ 判分设计：`0%` vs `-20%` 的含义

周测回顾页每个选项右侧的数字**不是**"有多少人选了它"，而是**该选项的得分权重**：

显示	含义
`100%` + "Correct answer"	正确选项（单选）
`0%`	**near miss（近似项）**——选了不扣分，但也不给分
`-20%`	**硬错项**——选了要倒扣
多选：`25%` / `33.33%` / `50%`	正确选项**均分**该题总分（3 项正确 → 各 33.33%）

关于单题负分：周测**确实会出现单题负分**——`week5/2.png` 右上角逐字为 `-2 / 10`（单选，选中 −20% 项）。但也观察到 `week2/8.png` 勾了两个各 −20% 的项却显示 `0 / 10`。**逐题扣分的显示规则不完全一致，不必深究**；与考试相关的是：**期中按 `requirement.txt` 是"总分不为负"**。

这条对复习的价值：`0%` 的选项就是老师心目中的"**最接近的干扰项**"——它精确标出了**这题真正考的那个区分点**。复习时优先看这些 near miss，它们告诉你"差一点就对了"的地方长什么样。

识别标记的语义（千万别混淆）：

```
Correct answer  →  绿色 + 100%（或该题均分%）    →  这才是答案
Incorrect: + 粉红高亮 + ☑                        →  这是「你自己选错的项」→ 不是答案
```

> ⚠️ 这些百分比**只在回顾页可见**，考试时看不到，不能当场上线索用——价值在**平时复习**。
> 📌 附带观察：页面顶部的 `Partial and negative credit` 蓝色横幅只出现在**有扣分风险**的题上；满分题的回顾页没有该横幅。

2.3 ⭐ 八大干扰项手法

手法 1 — API 属性/方法名近形混淆

`doc.ents` ✅ vs `doc.entities` ✗ · `entity.text` ✅ vs `entity.label_` ✗ · `label_` ✅ vs `label` ✗ · `get_scores` ✅ vs `get_score` ✗ · `get_feature_names_out` ✅ vs `get_features_names_out` ✗ · `f1_score` ✅ vs `f1_Score` ✗ · `most_similar` ✅ vs `similar` ✗ · `lda_mdel` ✗

手法 2 — 函数名 vs 参数名互换

`nltk.wsd.lesk`（函数名）被四个参数名 `sentence` / `target_word` / `tokens` 包围。

手法 3 — 方向反转（最高频 · 多选重灾区）

把某个关系的方向写反：

- 训练目标：**maximize** log-likelihood 写成 "minimize"
- negative sampling：**maximize** same-context similarity → 写成 "minimizes … same context and maximizes in different context"
- 指标侧重：**ROUGE 侧重 recall、BLEU 侧重 precision** → 写成对调
- LSA vs pLSA：**pLSA 更可解释** → 写成 "LSA provides more human-interpretable topics"

手法 4 — 概念/模型归属错置

把 A 的特性安到 B 上：

- GloVe 的 `f(x)` 加权动机 → 说成 Word2vec 的 context matrix
- PCA 的成分 → 说成 LDA 流程的一部分
- RBF kernel → 说成 Linear Regression "**always** uses"
- "dictionary forms"（lemmatization）→ 说成 stemming 的作用
- "semantic structure" → 说给 NER（实际属 DP）
- dependency parsing 里 **head ↔ modifier 对调**
- POS tag 定义 → 说成 "dependent relations to other tokens"
- LSTM **forget gate 用 ReLU、input gate 用 tanh 先决定**
- GNN 里 **node embeddings → 写成 "edge embeddings"**
- ToT ↔ GoT 结构说反

手法 5 — 算法族 / 机制误判

- **CRF 是 discriminative** → 写成 "belongs to a group of **generative** models"
- **ELM 不做 backpropagation** → 写成 "follows the traditional neural networks style of backpropagation to continuously update"
- **Naïve Bayes 用 Bag-of-Words、丢弃 syntax** → 写成 "considers the syntax and structure of words"
- **LayerNorm 逐 feature 维度** → 写成 "across different samples in a batch"（那是 **BatchNorm**）

手法 6 — 数值/比例的语义偷换

- `n_components=0.95` 是**累计解释方差比例** → 被说成"降到 95% 的特征数" / "选 95 个主成分" / "对 95% 的数据集做 PCA"
- `n_clusters` vs `n_init` vs `max_iter`
- `lr`（学习率）→ 说成层数 / batch size / sequence length

手法 7 — 绝对化措辞

出现 **"only" / "always" / "for every" / "does not require" / "never"** 的选项，**默认怀疑**：

- "Parameter estimation **can only** be performed using Quasi-Newton methods such as L-BFGS" ❌（实际 SGD / Quasi-Newton / L2 都可）
- "Linear Regression model **always** uses the Radial Basis Function kernel" ❌
- "The **only way** that POS tagging may be achieved is using entity labeling" ❌
- "CREATE new labels for **every** unseen document" ❌

手法 8 — 默认行为 / 遗漏步骤 / 顺序

- `get_feature_names_out()` **默认按字母序**，不是出现顺序 → 选项给"词对了但顺序错"的 near miss
- `TfidfVectorizer()` **默认不去停用词**；要显式传 `stop_words='english'`
- Pipeline 少做一步（漏 lemmatization / 漏 pos_tag）或顺序倒置
- **ARGM 修饰语被当成核心论元**（SRL 题）

2.4 常见考法（按周）

周	主流考法
W1	"这段代码在干什么"（正则 / stemming / lemmatization / stopwords）+ 补全 pipeline
W2	补全 NER 提取代码 + 理论多选（NER / POS / DP / WSD 的辨析）+ **反选题**
W3	代码**默认行为**预测（TfidfVectorizer 词表顺序、stop_words、PCA 的浮点语义）+ SRL 角色映射
W4	多选概念辨析：topic modeling / CRF / 评估指标 / 词嵌入 / 分类器
W5	Keras/PyTorch 代码：输出层设计、`compile` 的 loss、`lr` 含义、gradient clipping、GNN
W6	多选概念题：regularisation、Transformer 组件、ViT、LSTM/GRU 门、encoder-decoder
W7	读 attention 代码问"为什么这么做"——softmax / $\sqrt{d_k}$ / LayerNorm 的作用

三、逐周知识点精讲 + 真题印证

> 本节的写法：**先讲知识点，再用真题印证**。每个知识点都告诉你"它是什么 → 真题怎么考它 → 陷阱在哪"。左侧表格是答案速查，正文是理解。带 ⭐ 的是高频考点。

Week 1 — Preprocessing（预处理）· 5 题 · 单选

⭐⭐ 知识点 1：Stemming vs Lemmatization（词干化 vs 词形还原）

这是 W1 最核心的一组概念，也是**全课程最高频的对立考点**。

	Stemming	Lemmatization
做什么	砍掉词缀，得到 **root form（词根）**	还原到 **dictionary form（词典原形）**
结果	可能不是真词（`easily` → **`easili`**）	一定是真词（`running` → `run`）
工具	`PorterStemmer().stem(w)`	`WordNetLemmatizer().lemmatize(w, pos=)`
依据	规则（rule-based）	词典 + **词性 pos**

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
PorterStemmer().stem('easily')                      # 'easili'  ← 不是真词
WordNetLemmatizer().lemmatize('running', pos='v')   # 'run'
WordNetLemmatizer().lemmatize('running')            # 'running' ← 不传 pos，默认当名词，还原失败!
```

> **真题印证（W1-Q1）**："`PorterStemmer` 这段代码的主要目的是什么？"
>
> **正确答案 B**：`To convert the words to their root forms.`
>
> **陷阱**：选项 A 说成 "**dictionary forms**"（词典原形）——那是 lemmatization 的活。该选项标 `0%`，是老师心目中的**最接近干扰项**。
>
> **Feedback 原文**：Stemming converts word to root form while lemmatization is for dictionary form.

> **真题印证（W1-Q2）**：`lemmatize('running', pos='v')` 在干什么？
>
> **正确答案 B**：`To find the base form of the verb 'running'`
>
> **陷阱**：选项 D 说成 "perform **speech tagging** on 'running'" —— `pos='v'` 是**输入参数**（告诉它按动词还原），不是在做词性标注。**这个"参数 vs 任务"的偷换是老师惯用手法。**

知识点 2：Tokenization（分词 vs 分句）

```
word_tokenize(text)   # 分词 → ['The', 'quick', 'brown', ...]
sent_tokenize(text)   # 分句 → ['The quick brown fox.', 'The dog slept.']
text.split()          # 粗暴按空格切，标点会黏在词上
```

真题印证（W1-Q4）：下面这段代码在做什么？

```
tokens = word_tokenize(text)
filtered_words = [w for w in tokens if w not in stopwords.words('english')]
```

正确答案 A：`Removing stopwords from the text.`

陷阱：选项 C 说成 "Tokenizing the text into **sentences**"（标 `0%`，near miss）——代码里明明白白是 `word_tokenize`（**词**），不是句子。**一字之差。**

⭐⭐ 知识点 3：正则表达式清理文本

```
re.sub(r'[^\w\s]', '', text)   # 去标点：匹配「不是词字符、也不是空白」的字符
```

模式	含义	陷阱
`\w`	词字符（字母/数字/下划线）	—
`\s`	空白	—
`\d`	数字	—
`\W`	**非**词字符	⚠️ **它同时会吃掉空格**
`[^\w\s]`	非词字符 **且** 非空白	✅ 精确命中标点

真题印证（W1-Q5）："用哪个正则配合 `re.sub` 去掉所有标点？"

正确答案 D：`'[^\w\s]'`

陷阱：选项 B `'\W+'` 标 `0%`（near miss）—— `\W` 看起来对，但它**会把空格也删掉**，而 `re.sub` 第二个参数是 `''`（删除而非替换为空格），所以词会全黏在一起。

Feedback 原文：`'\W+'` matches non-word characters but would also remove spaces.

知识点 4：Pipeline 的步骤与顺序

一段完整预处理的标准顺序：

```
原始文本 → sent_tokenize（分句）→ word_tokenize（分词）→ lemmatize（还原）→ pos_tag（词性标注）
```

真题印证（W1-Q3）：补全"先分句、再逐词还原、最后做词性标注"的三行代码。

正确答案 E：

```
Line 1: sentences = sent_tokenize(text)
Line 2: lemmatizer = WordNetLemmatizer()
Line 3: tagged = [pos_tag([lemmatizer.lemmatize(word) for word in word_tokenize(sentence)])
for sentence in sentences]
```

陷阱：A/B 漏了 lemmatize；D 漏了 pos_tag；C 顺序倒置（先整篇分词，丢了句子边界）。**数清题目要求几步**是解这类题的关键。

Feedback 原文：`Near misses: did not do lemmatization / did not break into sentences first, not a major issue / did not to pos tagging`

W1 必跑练习（对应 `weeek1/Week1MCQ.md`）：

```
words = ["running", "runner", "ran", "easily", "fairness", "isn't"]
[PorterStemmer().stem(w) for w in words]
# → ['run','runner','ran','easili','fair','isn't']   ← 注意 easily→easili、fairness→fair

[WordNetLemmatizer().lemmatize(w, pos='v') for w in words]
# → ['run','runner','run','easily','fairness',"isn't"]  ← 只对动词形态生效
```

Week 2 — Linguistic Features（语言学特征）· 10 题

⭐⭐ 知识点 1：spaCy 的对象层级与属性

这是 W2 丢分的最大来源——属性能不能访问，取决于你在哪一层。

```
Doc（整篇）
 ├── doc.ents          → 实体 Span 列表
 ├── doc.sents         → 句子 Span 列表
 └── Token
      ├── token.text    = token.orth_
      ├── token.pos_    粗粒度词性（DET / NOUN / VERB）
      ├── token.tag_    细粒度词性（DT / NN / VBZ）
      ├── token.dep_    依存关系（nsubj / dobj / prep）
      ├── token.head    父节点 Token
      └── token.children  子节点（生成器，要 list()）

Span / Entity
 ├── span.text
 ├── span.label_       实体或依存标签
 └── span.ents         ← 只有 Doc/Span 有，Token 没有
```

⚠️ **三条铁律**：

1. **下划线不能省**：`label_` / `pos_` / `dep_` / `orth_`。写成 `label` / `pos` / `dep` 拿到的是 hash 整数，不是文本。
2. `label`（实体类型）**在 Span 上，不在 Token 上**。
3. `sent.ents` **才有实体列表**，不能写成 `sent.ents.label_`（那是跨层取属性）。

真题印证（W2-Q1）："打印每个命名实体的文本"

正确答案 C：`print([entity.text for entity in doc.ents])`

陷阱：`doc.entities` ✗（不存在的属性）、`doc.ents.text` ✗（`doc.ents` 是列表，没有 `.text`）、`entity.is_ent` ✗、`entity.label_` ✗（那是标签不是文本）。

真题印证（W2-Q2）："抽取含组织（ORG）的句子"

正确答案 B：

```
if "ORG" in [ent.label_ for ent in sent.ents]:
    sentences_with_org.append(sent)
```

陷阱：选项 E 写成 `if sent.ents.label_ == "ORG"`（**跨层取属性**，`sent.ents` 是列表）；选项 C 在 `sent` 而不是 `sent.ents` 上找实体；选项 D 逻辑对但用 `ent.label`（少下划线）且会把同一句重复 append。

Feedback 原文：`Near misses: one checks for label in sent instead of sent.ents / another will append the same sentence multiple times if it contains more than one "ORG" entity.`

知识点 2：Stopword 过滤与去重

```
from nltk.corpus import stopwords
filtered = [w for w in word_tokenize(text) if w not in stopwords.words('english')]
set(lemmas)   # 去重——没有 unique() 这个内置函数
```

真题印证（W2-Q3）：还原词形并**打印不重复的** lemma。

正确答案 E：

```
Line 1: lemmatizer = WordNetLemmatizer()
Line 2: lemmas = [lemmatizer.lemmatize(word) for word in word_tokenize(text)]
Line 3: print(set(lemmas))
```

陷阱（三重）：忘了 `word_tokenize`（会按字符遍历）；忘了去重；虚构的 `nltk.Lemmatizer()` 和 `unique()`。**每个干扰项只差一个环节**。

Feedback 原文：the sentence need to be tokenized first, the set function gives the distinct words.

知识点 3：WSD（词义消歧）

问题：一个词有多个义项，怎么判断当前语境用的是哪个？ **Dictionary-based 方法（Lesk 算法）**：拿词典里每个**义项的定义（gloss）**去和**上下文**比对，重叠最多的就是答案。

```
from nltk.wsd import lesk
lesk(context_sentence, target_word)   # 返回最可能的 Synset
```

> **真题印证（W2-Q4）**：`from nltk.wsd import ___` 里被遮住的算法名是什么？
>
> **正确答案 A**：`lesk`
>
> **陷阱**：干扰项 B `sentence`、D `tokens`、E `target_word` **全是 `lesk()` 的参数名**。老师在考"你要的是函数名还是参数名"。
>
> **Feedback 原文**：It is the Lesk algorithm. Sense definition is after the Lesk algorithm is being performed.

> **真题印证（W2-Q6，多选）**：dictionary-based WSD 的特点？
>
> **正确答案 B, C, D**：`Does not require labeled training data` / `Uses gloss definitions` / `Measures overlap with surrounding context`
>
> **陷阱**：选项 A "Identifies one sense per sentence" 把 Lesk 的行为过度泛化。

知识点 4：NER / POS / DP 三者的辨析

	全称	做什么
NER	Named Entity Recognition	从非结构化文本抽出**实体**（人名/地点/公司/日期）
POS	Part-of-Speech tagging	给每个词标**词性类别**
DP	Dependency Parsing	分析词与词的**依存关系**，形成**有向树**

关键区分：

- DP 里 **head（主成分）** 与 **modifier（修饰成分）** 的关系——**主成分叫 head，依附的叫 modifier**
- DP 的标签（subject / object / modifier）标在**边（edge）**上
- `en_core_web_sm` **可以**做依存解析
- DP **没有**"固定大小为 4 的滑动窗口"这种说法

> **真题印证（W2-Q8，反向多选）**："选出关于 dependency parsing 的 **INCORRECT** 陈述"
>
> **正确答案 A, D**：
>
> - **A**：`The words that are the main components are called modifiers while the words that depend on them are called heads.` ← **head/modifier 说反了**，是假命题 ✓
>
> - **D**：`It analyses the sequence of words of a sentence within a fixed sliding window size of 4.` ← 无中生有，是假命题 ✓
>
> ⚠️ **你当时选了 B 和 C —— 它们其实都是真命题**（B 说"每个关系被赋予 label 如 subject/object/modifier"、C 说"可以用 en_core_web_sm 做依存解析"，Feedback 里两句话逐字出现）。**在问 INCORRECT 的题里选真命题，是最典型的反向题陷阱。**

真题印证（W2-Q9，反向多选）："选出关于 POS tagging 的 **INCORRECT** 描述"

正确答案 A, D：

- **A**：把 POS tag 的定义说成"指向其他 token 的依赖关系"（那是 **dependency relation**）✓

- **D**：`The only way that POS tagging may be achieved is using entity labeling.` ← **绝对化措辞** "only way"，且实际可用 HMM/Viterbi ✓

⚠️ **你选了 B 和 E —— 同样都是真命题**（Viterbi 优化标签序列、英语 5 大词性，Feedback 原文支持）。

真题印证（W2-Q10，多选）：NER 的正确描述?

正确答案 A, C, D, E（各 25%）：Google 搜索用它抽取人物/地点信息 / 从非结构化文本抽地点公司日期 / 答案检测 / 低阶信息检索

陷阱：选项 B 用**否定句式**"sentiment analysis **does not require** NER"反转 Feedback 原文。

真题印证（W2-Q7，多选）：NER 与 DP 怎么比较?

正确答案 B, E（各 50%）：都用 spaCy 实现且可可视化 / NER 适合答案检测、DP 适合标注主宾语

陷阱：选项 C 把 HMM 的 emission/transition probability 术语硬套到 NER→DP 关系上。

spaCy 速查卡（建议背下来）：

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple is looking at buying U.K. startup for $1 billion.")

[(e.text, e.label_) for e in doc.ents]     # [('Apple','ORG'), ('U.K.','GPE'), ('$1
billion','MONEY')]
[(t.text, t.pos_, t.tag_) for t in doc]    # ('the', 'DET', 'DT') ...
[(t.text, t.dep_, t.head.text) for t in doc]
spacy.explain('_SP')                       # 'whitespace'（注意是 explain 不是 explains）
```

Week 3 — 文本向量化与语义分析 • 5 题 • 单选

⭐⭐ 知识点 1：TF-IDF 的默认行为

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()                      # 默认【不】去停用词
vectorizer = TfidfVectorizer(stop_words='english')  # 显式去除

X = vectorizer.fit_transform(corpus)
vectorizer.get_feature_names_out()                  # ⚠️ 返回【字母序】，不是出现顺序
```

⚠️ **两个默认行为是本知识点的全部考点：**

1. `get_feature_names_out()` **按字母序排列**
2. **不传** `stop_words` **就不会去停用词**

真题印证（W3-Q2）：

```
corpus = ["The cat sat on the mat", "The dog sat on the rug"]
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
```

正确答案 A： `['cat', 'dog', 'mat', 'rug', 'sat']`

陷阱： `the` 和 `on` 被去掉（B、E 项把它们留下了）；`sat` **不是**停用词要保留（D 项把它删了）；**C 项词全对但按出现顺序排列，是** `0%` **的 near miss**。

Feedback 原文： The stop words 'the' and 'on' are removed ⋯ Remaining words are **alphabetically sorted**.

Near Miss: `['cat','sat','mat','dog','rug']` : Correct words but are **in the order of appearance**. However, the code will arrange in **alphabetical order**.

真题印证（W3-Q5）：

```
corpus = ["apple banana apple", "banana orange"]
X = TfidfVectorizer().fit_transform(corpus)
print(len(vectorizer.get_feature_names_out()))
```

正确答案 C： `3`（apple / banana / orange——**去重后的词表大小**）

陷阱： 5 = token 总数（把 apple 重复计数）；2 = 误以为 apple 因高频被当停用词剔除。

⭐ 知识点 2：PCA 的 `n_components` —— int 还是 float，含义完全不同

```
PCA(n_components=3)      # int  → 保留【前 3 个】主成分
PCA(n_components=0.95)   # float → 保留累计【解释方差达到 95%】的成分
```

取值类型	含义	结果维度
`int`	主成分**个数**	恰好该个数
`float` ∈ (0,1)	累计**方差比例**	不固定，取决于数据

真题印证（W3-Q3）：`PCA(n_components=0.95)` 意味着什么？

正确答案 A：`Retain components that contribute to 95% of the variance.`

陷阱：B 项说成"降到 95% 的**特征数**"（`0%` near miss，你的错选）；E 项说成"选 **95 个**主成分"（`0%`）；D 项说成"对 95% 的数据集做 PCA"。

Feedback 原文：Near misses: This choice misleadingly suggests a proportion of **original features**, not variance. It's a common misunderstanding to take the float value as a **direct count** of components.

知识点 3：Lemmatization 在 TF-IDF 之前的作用

真题印证（W3-Q1）：向量化之前先做 `tagged_lemma` 的好处是什么？

正确答案 B：`It reduces vocabulary sparsity by collapsing morphological variants into a canonical form.`

怎么理解：把 `run / runs / running / ran` 归并成一个 `run`，**词表变小、稀疏度下降**，向量更有意义。

陷阱：说成"去停用词"✗；说成"统一变成一般现在时"（`0%` near miss，过度窄化）✗；说成"加快 TF-IDF 收敛"✗。

⭐ 知识点 4：SRL（Semantic Role Labeling，语义角色标注）

问的是"谁对谁做了什么"——谓语（Predicate）+ 论元（Arguments）。

```
句子: Students learn NLP in NTU.

      ┌─────────┬─────────┬─────────────────┐
      | Agent   |Predicate|   Patient       |
      |Students |  learn  |    NLP          |
      └─────────┴─────────┴─────────────────┘
                  (in NTU 是 ARGM 修饰语，不是核心论元)
```

依存标签 → 论元角色的映射（必背）：

依存标签	论元	角色
`nsubj`	**ARG0**	**Agent**（施事，谁做的）
`dobj`	**ARG1**	**Patient**（受事，对谁做的）
`prep`	**ARGM-***	**修饰语**（时间/地点等，**不是核心论元**）

真题印证（W3-Q4）：`"Students learn NLP in NTU."` 的 Agent / Patient / Predicate 分别是什么？

正确答案 D：`Agent: Students Patient: NLP Predicate: learn`

陷阱：选项 E 把 `in NTU` 当成 Patient（`0%` near miss，**你的错选**）——它走的是 `prep → ARGM-NTU` 分支，是**修饰语**不是受事。

Predicate 就是句子的 root verb。

Week 4 — 概念辨析专题 • 5 题 • 全多选

> W4 全部是多选概念题，考的**不是代码而是理解**。多选 + 倒扣，所以"只选有把握的"。

⭐⭐ 知识点 1：主题模型家族（LSA / pLSA / LDA）

模型	原理	关键特性
LSA	对 **TF-IDF 文档-词矩阵**做 **SVD** 降维	能捕捉语义关系；**矩阵填 TF-IDF 分数**
pLSA	把文档生成看作**主题的概率混合**	**比 LSA 更可解释**（主题 = 词上的概率分布）
LDA	概率生成模型，主题分布 + 词分布	**不考虑词的上下文**，捕捉不到深层语义

> **真题印证（W4-Q1，多选）**：如何描述这些主题模型算法?
>
> **正确答案 B, C, D**：
>
> - **B**：`pLSA views the generation of a document as a probabilistic mixture of topics` ✓
> - **C**：`LDA treats words independently of their context in a document, hence deep semantics are not captured` ✓
> - **D**：`The document-term matrix in LSA is typically populated with TF-IDF scores for each word` ✓
>
> **陷阱**：选项 A 说 "**LSA** provides more human-interpretable topics compared to pLSA"（**正好说反**——Feedback 明说 "pLSA is more interpretable"）；选项 E 把 **PCA** 的成分说成 LDA 流程的一部分。
>
> **Feedback 原文**：LDA … does not consider meaning or context of words … **pLSA is more interpretable** as it explicitly defines topics as probability distributions over words.

⭐⭐ 知识点 2：CRF（条件随机场）

CRF 是什么：用来预测**序列标签**的模型，会考虑**标签之间的依赖**（比如 POS 标注、NER）。

```
CRF 关键性质（全是要考的）：
  ✓ discriminative（判别式模型）—不是 generative
  ✓ 训练目标：maximize log-likelihood（最大化，不是最小化）
  ✓ 参数估计方法：SGD / Quasi-Newton / L2 正则 —— 不止一种
  ✓ 可捕捉 lexical（词本身）/ syntactic（词性）/ contextual（邻词）特征
  ✓ 可与 Bi-LSTM / Bi-GRU 组合做多语言 NER
```

> **真题印证（W4-Q2，反向多选）**："哪些**不能**描述 CRF 的工作原理？"（**问的是 DO NOT**）
>
> **正确答案 B, C, E**（这三个都是**假命题**）：
>
> - **B**：`The goal during training CRF is to **minimize** the log-likelihood` ← 应为 **maximize** ✓
> - **C**：`CRF belongs to a group of **generative** models that create new labels for every unseen document` ← 应为 **discriminative** ✓
> - **E**：`Parameter estimation **can only** be performed using Quasi-Newton methods such as L-BFGS` ← **绝对化**，实际 SGD / Quasi-Newton / L2 都行 ✓
>
> ⚠️ **你只勾了 B、E，漏了 C（得 6.67/10）。** 选项 A、D 是**真命题**，选了会倒扣。

⭐⭐ 知识点 3：评估指标的分工

指标	衡量什么	侧重 / 关键点
Precision	预测为正的里有多少真的对	—
Recall	真实为正的里有多少被找出	—
F1	precision 与 recall 的**调和平均**	两者兼顾
BLEU	n-gram **重合精度**	**侧重 precision**；主用于**机器翻译**
ROUGE	n-gram **召回**	**侧重 recall**；主用于**摘要**
METEOR	含 **chunk penalty**，考虑词序	比 BLEU 更细
AUC-ROC	识别正例 vs 误判负例的 **tradeoff**	与阈值无关的整体排序能力

真题印证（W4-Q3，多选）：关于评估指标哪些说法是对的？

正确答案 A, B, C, E（各 25%）：METEOR 有 chunk penalty / BLEU 基于 n-gram 重合 / F1 是调和平均 / AUC-ROC 是 tradeoff

陷阱：选项 D 把 **ROUGE 与 BLEU 的 precision/recall 侧重对调**——这是唯一扣分项。

Feedback 原文：**ROUGE score is focused on the recall** rather than the precision of the n-grams. **BLEU is indicative of the precision** of matching n-grams.

⚠️ **你只勾了 C、E（得 5/10），漏了 A、B。** 多选要覆盖全，四个正确项平均才拿满分。

知识点 4：Word2vec vs GloVe

	Word2vec	GloVe
训练方式	**迭代**训练，滑动窗口逐词推进	基于**全局共现统计**
窗口	**sliding window**（滑动）	**fixed window size**（固定）
特色机制	**Negative sampling**、Softmax	`f(x)` **加权函数**

```
# GloVe 的 f(x)：给每个共现按频率加权
#   目的：罕见/噪声共现（rare & noisy co-occurrences）信息量低，要压低它们的权重
# Word2vec 的 negative sampling：
#   让【同上下文】的词相似度最大化，【不同上下文】的相似度最小化
# Word2vec 用 Softmax 的直觉：
#   上下文相似的词，向量表示也应该相似
```

真题印证（W4-Q4，多选）：Word2vec 与 GloVe 里关键函数的作用?

正确答案 A, D（各 50%）：

- **A**：`The objective function in GloVe incorporates a function that prevents the model from being influenced by common word pairs`（就是 `f(x)`）✓

- **D**：`The intuition behind Softmax in Word2vec is that words in similar contexts will have similar vector representations` ✓

陷阱（三重）：

- 选项 B 把 **GloVe 的 rare/noisy co-occurrence 动机安到 Word2vec 的 context matrix** 上（**你的错选**）

- 选项 C 把 negative sampling 写成 "**minimizes** … same context"（**方向反了**）

- 选项 E 把 GloVe 的 **fixed** window 说成 **variable-sized sliding window**

⭐⭐ 知识点 5：四个分类器的原理

模型	一句话原理	常见错误说法
Naïve Bayes	**Bag-of-Words，丢弃语法与结构**	✗ "考虑 syntax 和 structure"
SVM	找**最大间隔超平面**分隔数据	✗ 说成"图形化方法"这类模糊表述
GP（高斯过程）	把**整个函数**建模为随机变量，用 **mean + covariance** 刻画	✗ 说成只建模数据点
ELM（极限学习机）	单隐层前馈网络，**不做 backprop**，权重**只更新一次**	✗ "遵循传统神经网络的反向传播持续更新权重"
Linear Regression	线性模型	✗ "**always** 用 RBF kernel"

真题印证（W4-Q5，多选）：哪些准确描述了各分类器的工作原理?

正确答案 A, B（各 50%）：

- **A**：`SVM finds the hyperplane that best separates data points of different classes in a high-dimensional space` ✓

- **B**：`GP models the entire functions as random variables, characterized by mean and covariance` ✓

⚠️ **你选了 A、D、E（得 1/10）** —— D 和 E 都是错的：

- **D**：`Linear Regression model **always** uses the RBF kernel`（**绝对化** + RBF 属 SVM/核方法）

- **E**：`Naïve Bayes model considers the **syntax and structure** of words`（**正好反了**，NB 用 Bag-of-Words 丢弃语法）

Feedback 原文：The Naïve Bayes algorithm adopts a **Bag of Words** approach — the syntax and structure of words are **discarded**. … **ELMs do not backpropagate**. The weights are **only updated once**.

Week 5 — 神经模型与训练技巧 • 5 题 • 单选

⭐ 知识点 1：Gradient Clipping（梯度裁剪）

解决什么问题：RNN/LSTM 里梯度可能**爆炸（exploding）**，训练发散。 **怎么做**：把梯度的**范数（norm）**上限卡住。

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)   # 按【范数】裁剪
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5) # 按【值】裁剪
```

函数	裁剪依据	参数
`clip_grad_norm_`	梯度**范数**	`max_norm`
`clip_grad_value_`	梯度**值**	`clip_value`（**float，不是字符串**）

> **真题印证（W5-Q1）**：`clip_grad_norm_` 在这段代码里的主要目的是什么?
>
> **正确答案 B**：`To prevent exploding gradients by capping the gradient norm.`
>
> **陷阱**：说成"把梯度限制到 [0,1] 区间"✗（它限制的是 **norm**，不是每个梯度的取值范围）；说成"加速收敛"✗；"稀疏化梯度"✗；"去掉小梯度"✗。

知识点 2：Keras 输出层与 loss 的任务匹配

任务	输出层	loss
二分类	`Dense(1, activation='sigmoid')`	`binary_crossentropy`
多分类	`Dense(n_classes, activation='softmax')`	`categorical_crossentropy`
回归	`Dense(1)`（无激活）	`mean_squared_error`

> **真题印证（W5-Q3）**：二分类情感分析的输出层该写什么?
>
> **正确答案 E**：`model.add(Dense(1, activation='sigmoid'))`
>
> **陷阱**：选项 D `Dense(2, activation='softmax')` 标 `0%`（near miss，**你的错选**）——那是**多分类**的做法；二分类只需要 1 个神经元配 sigmoid。选项 A 用 `relu`（输出无界，不是概率）。
>
> **Feedback 原文**：Near misses: One is suitable for **multi-class** classification, not binary. `'relu'` activation is not ideal for a binary classification output.

> **真题印证（W5-Q4）**：多分类任务该用哪行 `compile`?
>
> **正确答案 E**：`model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])`
>
> **陷阱**：选项 C 是最强干扰——optimizer 和 metrics 全对，**只有 loss 错成** `binary_crossentropy`。

知识点 3：学习率是什么

```
optim.Adam(model.parameters(), lr=0.001)
RMSprop(lr=0.01)
```

> **真题印证（W5-Q5）**：`RMSprop(lr=0.01)` 里的 `lr=0.01` 起什么作用？
>
> **正确答案 C**：`It determines the speed of convergence during training.`
>
> **陷阱**：四个干扰项**全在考"参数归属"**——说成隐藏层数 / 激活函数 / batch size / 序列长度。`lr` 就是 **learning rate（学习率）**，控制每次更新的步长。

知识点 4：GNN 的 Message Passing（课堂短讲补充）

> ⚠️ 这部分**不在仓库的课件和笔记里**，来自课堂短讲。

```
class SimpleGNN(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='mean')   # 聚合方式：平均
        self.lin = torch.nn.Linear(in_channels, out_channels)
```

GNN 消息传递的核心三步：

1. **看邻居** —— 节点用**邻接矩阵（adjacency matrix）**查自己和谁相连
2. **收特征** —— 收集邻居的特征向量（embeddings）
3. **更新自己** —— 把邻居信息与自身特征合并，形成新表示

```
邻接矩阵 × 节点特征向量  →  把消息传给邻居  →  更新【node】embeddings
```

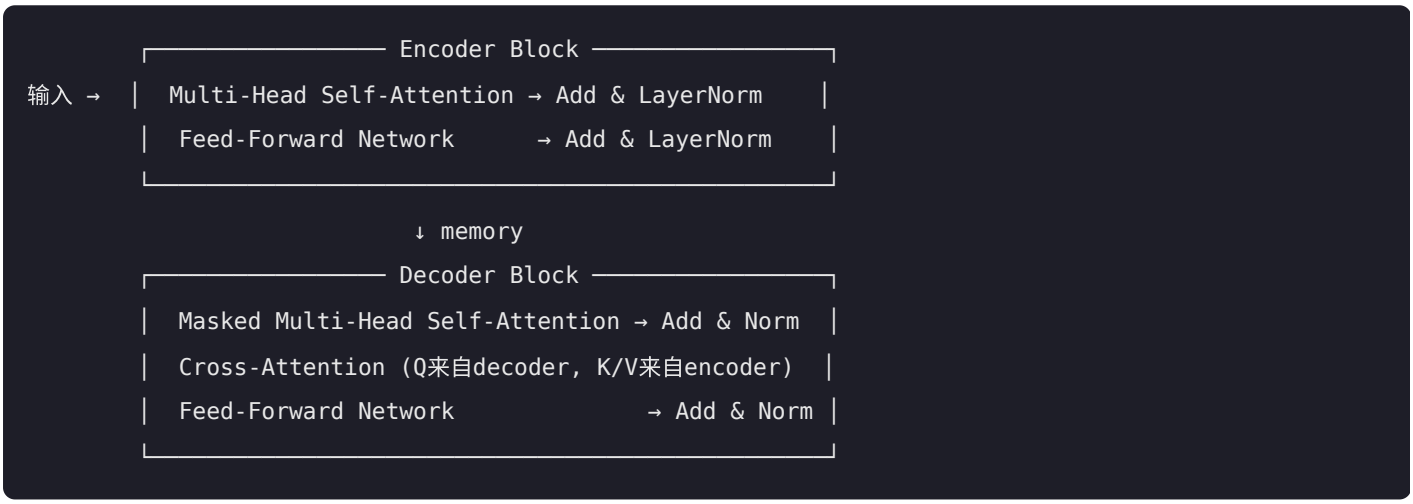
> **真题印证（W5-Q2）**：这段 `MessagePassing` 代码后续发生什么？
>
> **正确答案 C**：`Adjacency matrix is multiplied with node feature vectors to effectively pass messages to neighbours`
>
> **陷阱（三重）**：选项 A 说 "without any combination of self-loops"（**你的错选**，self-loops 恰恰常被加入以保留自身信息）；选项 D 把 **node** embeddings 写成 **edge** embeddings；选项 E 把 `aggr='mean'` 说成 maximum/summation。
>
> **Feedback 原文**：… Multiplying the adjacency matrix with the node feature vectors effectively passes messages to neighbours. … used to update the **node** embeddings (**NEAR MISS: should be "node", not "edge"**) … the screenshot indicates "mean" hence, taking the average … **not the maximum or summation**. Attention mechanism is not used for this particular example.

同周练习题补充（`Week6MCQ.md` / `Week8MCQ.md`）：

考点	必背
`nn.Embedding` 层	把 token 索引映射为稠密向量
改成 Bi-RNN	加 `bidirectional=True`
改成 LSTM	把 `nn.RNN` 换成 `nn.LSTM`
`optimizer.zero_grad()`	清空上一轮累积的梯度
K-Fold	`GridSearchCV` / `cross_val_score` **默认 cv=5**

Week 6 — Transformer 与正则化 · 5 题 · 全多选

⭐⭐ 知识点 1：Transformer 的完整结构



三个最容易搞混的点：

概念	正确理解	常见错误说法
Residual connection	把**输入直接加到输出**上	✗ "归一化输出" / ✗ "跑在 self-attention 之前"
它解决的梯度问题	**vanishing**（消失）	✗ "exploding"（爆炸）
Layer Normalisation	在 residual **之后**做归一化	✗ 与 residual 混为一谈
FFN sublayer	两个线性变换 + 中间激活	✗ 写成 "**feedbackward**" 网络

> **真题印证（W6-Q2，多选）**：关于 Transformer 各组件的正确描述?
>
> **正确答案 C, D**（各 50%）：masked multi-head self-attention 的定义 / parallel attention heads 同时捕捉不同关系
>
> **陷阱**：选项 B 把 **LayerNorm 的功能安给 residual connection**（你的错选），且把 vanishing 说成 exploding；选项 E 把 **feedforward 错拼成** `feedbackward`。
>
> **Feedback 原文**：There is no such thing as a "**feedbackward**" neural network sublayer. The correct term is a **feedforward** neural network (FFN) sublayer. ⋯ residual connection ⋯ add the original input of a layer directly to its output ⋯ mitigates the **vanishing** gradient problem. **Layer Normalisation** then normalises the output of each sublayer **post-residual connection**.

⭐⭐ 知识点 2：LSTM / GRU 的门控机制

LSTM 三道门：

门	用什么激活	作用
Forget gate	**sigmoid**	决定 cell state **丢弃**什么
Input gate	**sigmoid**（先决定）+ **tanh**（生成候选值）	决定**存入**什么新信息
Output gate	sigmoid	决定 cell state **哪部分输出**

GRU 两道门：

门	作用
Update gate	控制**保留多少**上一隐状态 + **加入多少**新候选状态
Reset gate	决定**遗忘多少**上一隐状态

> **真题印证（W6-Q4，多选）**：为什么要用这些门?
>
> **正确答案 B, C, D**（各 33%）：
>
> - **B**：output gate 决定 cell state 的哪部分用来产生当前输出 ✓
> - **C**：GRU 的 update gate 决定保留多少 + 加入多少 ✓
> - **D**：GRU 的 reset gate 决定遗忘多少上一隐状态 ✓
>
> **陷阱**：选项 A 说 forget gate 用 **ReLU**（应为 **sigmoid**）；选项 E 说 input gate 先由 **tanh** 决定（应为 **sigmoid 先决定，tanh 只生成候选值**）。**A 和 E 都是你的错选。**
>
> **Feedback 原文**：The forget gates on an LSTM decides what information is kept … passed into a **sigmoid** function. Input gates … A **sigmoid** layer first decides which value to update … A **tanh** function then scales the input between 1 and -1.

⭐ 知识点 3：Encoder-Decoder 与 Attention

标准的 attention 流程（W6-Q5 与 W7 都在考）：

```
1. 用 encoder 的 hidden states 作【keys】，decoder 当前 hidden state 作【query】
2. query · keys 算分数（点积，不是 concatenated sum）
3. softmax → 归一化的 attention weights（不是 mean pooling 得到）
4. 对 encoder hidden states 做【加权和】
5. 这个加权和作为 context 送给 decoder
```

组件	作用
Encoder	输入是 **embedding 向量**；用 self-attention 捕捉层次化表示
Decoder	**自回归（autoregressive）**生成；只能看**已生成**的输出，看不到未来

真题印证（W6-Q5，多选）：如何描述 encoder 和 decoder 的工作原理?

正确答案 A, B, E（各 33%）：decoder 自回归生成且考虑上下文 / decoder 只能关注已生成输出 / encoder 输入是 embedding 向量

陷阱（两条都是你的错选）：

- **C**：说 key 和 query "compute a concatenated sum" 送进 encoder（**实际是点积**）
- **D**：说 attention weights 来自 **mean pooling**（**实际来自 softmax**；mean pooling 根本不产生 attention weights）

知识点 4：正则化（Regularisation）

方法	做法	效果
Dropout	训练时**随机**忽略一部分神经元	防止 co-adaptation；像 ensemble；降 variance
Weight decay（L2）	loss 里加权重惩罚项	抑制大权重；防止过度依赖单个特征

真题印证（W6-Q1，反向多选）："选出关于正则化的 **INCORRECT** 陈述"

正确答案 B, D（各 50%）：

- **B**：`Regularisation significantly **increases** both the variance of a model and the resulting bias` ← 应为 **reduces variance**、基本不增 bias ✓
- **D**：`**Weight decay** refers to **randomly selecting neurons** and ignoring them` ← 那是 **Dropout** ✓

⚠️ **你选了 C 和 E —— 它们在 Feedback 里都是真命题**（C 说 weight decay 抑制依赖单个特征、E 说 dropout 像 ensemble 且防 co-adaptation）。**又一次栽在反向题的"选到真命题"上。**

Feedback 原文：Regularisation, significantly **reduces** the variance of the model, without substantial increase in its bias. **Dropout**: … randomly selected neurons … are ignored … **Weight Decay**: also known as **L2 regularisation** …

知识点 5：Vision Transformer（ViT）

ViT 特性	说明
如何处理图像	**切成 patch**，把每个 patch 当作一个 **token**
核心机制	**self-attention**，让图像**远近部分直接交互**
时间线	ViT（2020）**晚于** AlexNet（2012）
应用	是 **CLIP / DALL·E** 等多模态模型的 building block

真题印证（W6-Q3，多选）：关于 ViT 哪些说法是对的?

正确答案 A, C, D（各 33%，你全对，10/10）

陷阱：选项 B 考**时间顺序**（说 ViT 早于 CNN）；选项 E 说 "CNNs and ViTs **both** rely on convolution filters"（ViT 靠 patch + self-attention，不靠卷积）。

Week 7 — Attention 代码精读 • 5 题 • 单选

⭐⭐ 知识点 1：`scaled_dot_product_attention` 逐行精读

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)  # ① 算分 + 缩放
    if mask is not None:
        attn_scores = attn_scores.masked_fill(mask == 0, -1e9)           # ② 掩码
    attn_probs = torch.softmax(attn_scores, dim=-1)                      # ③ 归一化
    output = torch.matmul(attn_probs, V)                                 # ④ 加权和
    return output
```

四个操作的作用，一个都不能混：

操作	作用	常见错误说法
`/ math.sqrt(d_k)`	**缩放**：让分数处于**可比尺度**，防止数值过大	✗ "归一化使和为 1"（那是 softmax）✗ "降维" ✗ "掩码"
`masked_fill(mask==0, -1e9)`	**掩码**：屏蔽不该看的位置	—
`torch.softmax(..., dim=-1)`	**归一化**：转成**概率分布**	✗ 说成 dropout 机制 ✗ 说成梯度下降 ✗ 说成"对 Q,K 矩阵归一化"
`matmul(attn_probs, V)`	**加权和**	—

> **真题印证（W7-Q1）**：为什么要对 `attn_scores` 应用 softmax?
>
> **正确答案 E**：`To convert the attention scores into a probability distribution`
>
> **陷阱**：B 说成 dropout 机制；C 说成 gradient descent；A 说成"对 Q, K 矩阵归一化"（实际是对 **attn_scores** 沿 `dim=-1` 归一化）。

> **真题印证（W7-Q4）**：为什么要除以 `math.sqrt(Q.size(-1))`？
>
> **正确答案 A**：`To ensure the attention scores are on a comparable scale, preventing extremely large values.`
>
> **陷阱**：B 把 **softmax 的功能**（归一化使和为 1）挪给 √d_k 除法（标 `0%`，near miss）；C 说成降维；E 说成 masking。

> ⚠️ **注意两题的代码差异**：Q1 的版本用 `math.sqrt(self.d_k)`（类方法版）；Q4 的版本用 `math.sqrt(Q.size(-1))`（函数版）。**同一个公式的两种写法，老师拿来出两道题。**

⭐ 知识点 2：`DecoderLayer` 里的两种注意力

```
def forward(self, x, memory, src_mask, tgt_mask):
    x = self.self_attn(x, x, x, tgt_mask)        # Q=K=V=x → 自注意力
    x = self.cross_attn(x, memory, memory, src_mask)  # Q=x, K=V=memory → 交叉注意力
```

看 Q 和 K/V 是不是同一个东西：

	Q	K / V	含义
`self_attn`	target	target	**target 序列内部**的自注意力
`cross_attn`	target	**source（memory）**	**target 与 source 之间**的注意力

> 代码里两个模块的**构造完全相同**（都是 `MultiHeadAttention(d_model, num_heads)`），**区别只在 forward 传入的 Q/K/V 与 mask**。

> **真题印证（W7-Q2）**：`self.self_attn` 和 `self.cross_attn` 有何区别?
>
> **正确答案 A**：`self.self_attn is for self-attention within the target sequence, while self.cross_attn is for attention between target and source sequences.`
>
> **陷阱**：B/C 编造实现层差异（linear vs normalization、positional encoding vs embedding）；D/E 干脆否认差异（"都编码 source" / "同类型 attention"）。

⭐ 知识点 3：`nn.LayerNorm` —— 与 BatchNorm 的区别

```
LayerNorm：沿【feature 维度】归一化，对【每个 token】独立进行
BatchNorm：沿【batch 维度】归一化，跨样本进行
```

LayerNorm 保证两件事：zero-mean（零均值）AND unit variance（单位方差）——缺一不可。

> **真题印证（W7-Q5）**：`nn.LayerNorm(d_model)` 在 transformer 层里的作用?
>
> **正确答案 C**：`It normalizes activations across the feature dimension for each token.`
>
> **陷阱（两条都是你踩的）**：
>
> - 选项 B 说成"跨 batch 不同样本归一化"（`0%` near miss）——那是 **BatchNorm**
> - 选项 A 说成"只保证 zero-mean"（`0%`，**你的错选**）——**半对也算错**，Feedback 明说还要 unit variance
>
> **Feedback 原文**：Near Misses: Normalizing activations across different samples in a batch is incorrect because **BatchNorm** does this, while **LayerNorm works per feature dimension**. / **Partially correct**, but LayerNorm ensures both **zero-mean and unit variance**, not just mean correction.

知识点 4：ToT vs GoT（思维树 vs 思维图）

```
# Tree-of-Thoughts: 每个 node 挂多个 children —— 树状分支
root = {"state": problem, "children": []}
    node["children"].append(child)          # 层层分叉

# Graph-of-Thoughts: thoughts 是顶点，edges 表示依赖 —— 可互连的图
graph = {"nodes": [problem], "edges": []}
    graph["edges"].append(connect(node, t))  # 任意连接
```

	ToT	GoT
结构	**树**（有层级、有分支）	**图**（节点可互连）
thoughts 在哪	挂在**节点**上	**顶点是 thoughts，边表示依赖**
代码体现	`node["children"].append(...)`	`graph["edges"].append(connect(...))`

> **真题印证（W7-Q3）**：两种方法怎么区分？
>
> **正确答案 B**：`Thoughts are vertices and the edges show dependencies in Snippet 2 (GoT) while there are several children thoughts for each node in Snippet 1 (ToT).`
>
> **陷阱**：D 把两者说反（"GoT 是层级分支列表、ToT 允许互连"）；A、E 无中生有引入代码里根本没有的"有向/无向""weighted/unweighted edges"。

四、跨周高频陷阱 Top 15

#	陷阱	出现周	一句话破解
1	**方向反转**（maximize/minimize、precision/recall、same/different context）	W4	看到比较关系，**先确认箭头指向**
2	**API 带不带下划线 / 单复数 / 大小写**	W1,W2,W3	`label_` `pos_` `dep_` `get_scores` `f1_score` `get_feature_names_out`
3	**模型/概念归属错置**	W1,W2,W4,W5,W6,W7	问一句"这个描述是**谁**的特性"
4	**绝对化措辞**（only / always / every / does not）	W2,W4,W6	默认怀疑
5	**默认行为**（字母序 / 不去停用词 / 浮点语义）	W3	`get_feature_names_out()` 是**字母序**
6	**参数语义**（`n_components=0.95` 是方差比例，不是特征比例）	W3	区分 int（个数）与 float（比例）
7	**算法族误判**（CRF generative? ELM backprop?）	W4	CRF=discriminative；ELM 不 backprop
8	**stemming ↔ lemmatization**	W1	root form vs dictionary form
9	**word_tokenize ↔ sent_tokenize**	W1	词 vs 句
10	**激活函数归属**（sigmoid vs tanh vs ReLU）	W6	门用 sigmoid；候选状态用 tanh
11	**任务类型 ↔ loss/输出层匹配**	W5	二分类 `Dense(1,sigmoid)`；多分类 `softmax` + `categorical_crossentropy`
12	**LayerNorm ↔ BatchNorm**	W7	LayerNorm 按 **feature** 维度
13	**缩放 / 掩码 / 归一化 混淆**	W7	√d_k 缩放；masked_fill 掩码；softmax 归一化
14	**Pipeline 漏步骤 / 顺序倒置**	W1,W2	数清题目要求几步
15	**near miss 的"半对"**	W3,W7	"只保证 zero-mean" ✗——必须完整

五、考前冲刺清单

今晚（约 2.5 小时）

- [ ] **30 min** — 过 §二（出题习惯）与 §四（陷阱 Top 15）。**性价比最高**
- [ ] **30 min** — W1 + W2 真题（NLTK / spaCy API 精确拼写最容易丢分）
- [ ] **30 min** — W3 + W4 真题（**默认行为表**与**评估指标速查表**）
- [ ] **30 min** — W5 + W6 + W7（深度学习代码 + 概念）
- [ ] **20 min** — 遮住答案，重做**截图里标 INCORRECT 的题**（那是你实际错过的）

明早（约 40 分钟）

- [ ] **15 min** — 只看 §四 陷阱表 + §七 的「高概率考点 Top 10」
- [ ] **10 min** — 看 §三 W7 的 `scaled_dot_product_attention` 逐行精读
- [ ] **10 min** — 背 §三 W4 的评估指标表（BLEU/ROUGE/METEOR/F1/AUC-ROC）
- [ ] **5 min** — 检查考试物品：**一张空白 A4**、学生证、笔

答题时

- [ ] 单选题**一定不要空**
- [ ] 多选题按"不确定的选项不选"处理
- [ ] 填空题**逐字母检查拼写**（只填一个词）
- [ ] 遇到 "INCORRECT" 反选题——**先逐条判真假**再选

⭐ 必背拼写表（API / 术语）

这张表是填空题与代码题拼写考点的全部家底。 括号里是老师用来设坑的错误写法。

NLTK

正确	常见错误写法
`PorterStemmer`	`PorterStemer` / `PorterStemmer()` 类名大小写
`WordNetLemmatizer`	`nltk.Lemmatizer()`（不存在）
`word_tokenize`	`wordtok` / 与 `sent_tokenize` 混
`sent_tokenize`	`sent_tokenizer`
`stopwords.words('english')`	`stopword`（单数）
`pos_tag`	`POSTag`
`nltk.wsd.lesk`	`leks` / `lesk()` 的参数名（`sentence` / `target_word`）
`set(lemmas)` 去重	`unique(...)`（不存在）

spaCy

正确	常见错误写法
`spacy.load("en_core_web_sm")`	模型名拼错
`doc.ents`	`doc.entities`（不存在）
`entity.text`	`entity.txt`
`entity.label_`	`entity.label`（**少下划线**）
`token.pos_` / `token.tag_`	`token.pos` / `token.tag`
`token.dep_` / `token.head`	`token.dep`
`spacy.explain('_SP')`	`spacy.explains(...)`

scikit-learn

正确	常见错误写法
`TfidfVectorizer`	`TFIDFVectorizer`
`CountVectorizer`	`CounterVectorizer`
`get_feature_names_out()`	`get_features_names_out()`（**多一个 s**）
`LatentDirichletAllocation`	`LatentDirichletAlloc`
`TruncatedSVD`	`TruncatedSVD()` vs `PCA` 混用
`PCA(n_components=...)`	`n_components` 拼错 / 拼成 `n_component`
`SVC(C=..., kernel=...)`	`SVM(C=...)`
`GaussianProcessClassifier(kernel=RBF(length_scale=...))`	`length_scale` 传给分类器
`KMeans(n_clusters=...)`	`n_cluster`（少 s）/ `n_init` 混淆
`MultinomialNB` / `GaussianNB`	`MultinomialNaiveBayes`
`metrics.roc_auc_score`	`metric.roc_auc_score`（**少 s**）
`f1_score`	`f1_Score`（**大写 S**）

gensim / 评估

正确	常见错误写法
`BM25Okapi(...).get_scores(...)`	`get_score`（**单数**）
`Word2Vec(..., sg=0)`	`sg=0` 是 CBOW，`sg=1` 是 Skip-gram
`glove_model.most_similar('word')`	`most_similar` 拼错 / `similar`
`sentence_bleu([reference], candidate)`	reference 忘了 list 包裹

Transformer / PyTorch

正确	常见错误写法
`feedforward`	**`feedbackward`**（真题明确考过的假术语）
`nn.LayerNorm`	`nn.LayerNormalization`
`nn.MultiheadAttention`	`MultiHeadAttention` 大小写（课件两种写法都有，以题目给出的为准）
`scaled_dot_product_attention`	`scaled_dotproduct_attention`
`masked_fill`	`mask_fill`
`clip_grad_norm_`	`clip_grad_norm`（**结尾下划线**）
`clip_grad_value_`	同上；`clip_value` 是 float 不是字符串
`torch.nn.utils`	`torch.nn.util`（少 s）

概念术语

正确	常见错误写法
stemming / lemmatization	`lemmatisation`（无英美歧义，但别拼错）
`root form`（stemming）	与 `dictionary form`（lemmatization）对调
Bag-of-Words	`bag of word`
gradient **exploding** / **vanishing**	两者对调（residual 治 **vanishing**）
precision / recall	对调（**BLEU=precision，ROUGE=recall**）
`zero-mean and unit variance`	只说 zero-mean（**半对=错**）
discriminative / generative	CRF 是 **discriminative**
max**imize** log-likelihood	写成 minimize

六、应试策略

1. **先扫一遍题型**——数清单选 / 多选 / 填空各多少，估算时间（1h45min 通常够用）。
2. **单选不留空**：新规则下没有惩罚，全部填满。
3. **多选谨慎但有底**：总分不为负，比周测安全；仍按"只选有把握的"处理。
4. **代码题三问**：(a) **变量层级**是什么（doc / token / entity / sent）(b) 用的是**哪个 API** (c) 参数**属于谁**。
5. **"这段代码在干什么"**：先看 import 和最后的 print，别陷进中间细节。
6. **INCORRECT 反选题**：把每条当判断题标 T/F，再挑出 F 的。
7. **填空题**：先想中文意思 → 再想对应英文术语 → **逐字母拼写**。
8. **排除法优先级**：① 绝对化措辞 → ② 方向反转 → ③ 术语拼写可疑 → ④ 归属错置。

七、⭐ 期中考试预测

> ⚠️ **声明**：本节是基于 **40 道周测真题**逆向归纳的**预测**，不是提前获知的题目。目的是让你知道**该把时间花在哪**，以及**可能的题面长什么样**。考场上仍以实际题目为准。

7.1 预测依据

依据	说明
40 道周测真题	题型分布、干扰项手法、考点滚动规律
`requirement.txt`	明确了三种题型（单选 / 多选 / 填空）与判分规则
官方 lesson plan	范围 Week 1–6（归档含 W7，按 W1–7 准备）
老师本人的套路	同一知识点换代码写法反复考（如 W7 的 `self.d_k` vs `Q.size(-1)`）

7.2 题型结构预测

题型	估计占比	你的策略
单选（代码题 + 概念题）	**最多**（约 60%）	**绝不空着**——按 `requirement.txt` 单选不扣分
多选（理论辨析）	约 30%	题干会**明确标注**；不确定的项不选
填空（新题型）	约 10%	**逐字母拼写**；每空一个词

> **填空题型特别提示**（`requirement.txt` 明确提到但周测中没出现过）：
> - 答案**大小写不敏感**，但**拼写错了就没了**
> - **无英美拼写歧义**（不用纠结 behaviour/behavior）
> - **每空只填一个词或一个字母**，不能填词组
> - 最可能考：**算法名、API 名、术语**——即本册 §五 那张拼写表

7.3 各周考点出现概率

周	主题	预测概率	理由
W1	Stemming / Lemmatization / 正则	★★★★★	基础必考，几乎不可能缺席
W2	spaCy 属性（`label_` / `text` / `pos_`）	★★★★★	属性下划线是老师最爱考的细节
W3	TF-IDF 默认行为 / PCA `n_components`	★★★★☆	"库的默认行为"是 W3 的固定考法
W4	评估指标（BLEU/ROUGE/F1）/ 分类器原理 / CRF	★★★★☆	W4 全部是多选概念题，期中也必出
W5	Keras 输出层与 loss / 学习率 / 梯度裁剪	★★★★☆	"任务 ↔ 配置匹配"是固定考法
W6	Transformer 组件 / LSTM-GRU 门 / 正则化	★★★★★	重头戏，InterView 级必考
W7	Attention 代码逐行作用 / LayerNorm	★★★☆☆	代码精读，可能并入 W6 一起考

7.4 预测题（按真题风格编写）

📘 单选预测（7 道）

预测 1（W1，概率极高）

```
words = ["studies", "studying", "happy", "happily"]
stemmer = PorterStemmer()
print([stemmer.stem(w) for w in words])
```

A. `['studi', 'studi', 'happi', 'happili']`

B. `['study', 'study', 'happy', 'happily']`

C. `['studies', 'studying', 'happy', 'happily']`

D. `['studi', 'study', 'happi', 'happily']`

E. `['studies', 'study', 'happi', 'happili']`

> **答案 A**。PorterStemmer 是规则裁剪：`studies→studi`、`studying→studi`、`happy→happi`、`happily→happili`。
>
> **陷阱**：B 是 **lemmatization** 的结果（真词），是典型 near miss；E 混用了两种结果。

预测 2（W1/W2，概率极高）

```
doc = nlp("Singapore Airlines bought 10 Boeing planes.")
entities = [(e.text, e.label_) for e in doc.ents]
```

A. 打印 `(text, label)` 元组列表的 **正确** 写法

B. `[(e.text, e.label) for e in doc.ents]`

C. `[(e.text, e.ents) for e in doc.ents]`

D. `[(e.text, doc.label_) for e in doc.ents]`

E. `[e.text for e in doc.entities]`

> **答案 A（题干所示的写法）**。
>
> **陷阱**：B 少下划线（`label` 拿到的是 hash）；E 用了不存在的 `doc.entities`。**"下划线 + 属性归属"是必考项。**

预测 3（W3，概率高）

```
corpus = ["dog cat dog", "cat bird"]
vec = TfidfVectorizer()
X = vec.fit_transform(corpus)
print(vec.get_feature_names_out())
```

A. `['dog', 'cat', 'bird']`

B. `['bird', 'cat', 'dog']`

C. `['cat', 'dog', 'bird']`

D. `['dog', 'cat', 'bird', 'dog']`

E. `['bird', 'dog', 'cat']`

> **答案 B**。**字母序**：bird < cat < dog。
>
> **陷阱**：A 是**出现顺序**（W3 真题里就是这个 near miss 反复出现）；D 忘记去重；C/E 是其他错误排序。

预测 4（W3，概率高）

> `PCA(n_components=0.9)` 与 `PCA(n_components=9)` 的区别是什么？

A. 前者保留累计解释方差达 90% 的成分；后者保留前 9 个主成分

B. 前者保留 90% 的特征；后者保留 9 个特征

C. 两者等价

D. 前者保留前 9 个主成分；后者保留累计方差达 90% 的成分

E. 前者是对 90% 数据做 PCA；后者是对 9 个数据点做 PCA

> **答案 A。int = 个数；float = 方差比例。**
> **陷阱**：D 把两者对调；B 把"方差比例"说成"特征比例"。

预测 5（W5，概率高）

```
model = Sequential([
    LSTM(64, input_shape=(50, 300)),
    # ← 三分类情感分析的输出层
])
```

A. `Dense(1, activation='sigmoid')`

B. `Dense(3, activation='softmax')`

C. `Dense(3, activation='relu')`

D. `Dense(1, activation='softmax')`

E. `Dense(64, activation='softmax')`

> **答案 B。多分类 → n_classes 个神经元 + softmax。**
> **陷阱**：A 是**二分类**的写法（在 W5 真题里出现过）；C 用 relu 输出无界；D/E 神经元数错。

预测 6（W6/W7，概率高）

```
attn_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
attn_probs = torch.softmax(attn_scores, dim=-1)
```

> 这两行合起来完成了什么？

A. 先缩放分数防止数值过大，再用 softmax 转成概率分布

B. 先归一化成概率，再做缩放

C. 两行都在做归一化，效果重复

D. 第一行做掩码，第二行做加权和

E. 第一行降维，第二行做 dropout

> **答案 A。缩放 ≠ 归一化，两个操作目的不同。**
> **陷阱**：C 是"两者都在归一化"的混淆（W7 真题里 $\sqrt{}$d_k 那题就是靠这个设坑）。

预测 7（W1，概率中）

> 用 `re.sub` 去掉文本中的**所有数字**，正确的模式是？

A. `'\d+'`

B. `'[^\d]'`

C. `'\D+'`

D. `'[0-9]'`

E. `'\w+'`

> **答案 A**。`\d+` 匹配数字并删除。
>
> **陷阱**：D 只匹配**单个**数字（连续数字要多次替换）；C 是"非数字"（方向反了，会删掉所有字母）；B 是"单字符非数字"。

📗 多选预测（5 道）

预测 8（W6，概率极高）

> 关于 Dropout 和 Weight Decay，哪些说法是**正确**的?

A. Dropout 在训练时随机忽略一部分神经元

B. Dropout 可防止神经元之间的 co-adaptation

C. Weight decay 又被称为 L2 正则化

D. Weight decay 通过随机丢弃神经元来减少参数量

E. 正则化会显著提高模型的 variance 和 bias

> **答案 A, B, C**。
>
> **陷阱**：**D 把 Dropout 的描述安到 Weight Decay 上**（W6 真题原样考过）；E 把"降低 variance"说成"提高"（也是真题原句）。

预测 9（W6，概率极高）

> 关于 Transformer 的结构，哪些说法是**正确**的?

A. Residual connection 把输入直接加到输出上

B. Residual connection 负责归一化输出

C. Layer Normalisation 在 residual 之后进行归一化

D. Feed-forward sublayer 包含两个线性变换与中间激活函数

E. Residual connection 主要缓解 exploding gradient

> **答案 A, C, D**。
>
> **陷阱**：B 把 LayerNorm 的功能安给 residual；E 把 **vanishing** 说成 **exploding**。（B、E 都源于真题选项。）

预测 10（W6，概率高）

> 关于 LSTM 和 GRU 的门控，哪些说法是**正确**的?

A. LSTM 的 forget gate 使用 sigmoid 决定丢弃哪些信息

B. LSTM 的 forget gate 使用 ReLU 决定丢弃哪些信息

C. LSTM 的 input gate 先用 sigmoid 决定更新哪些值，再用 tanh 生成候选

D. GRU 的 update gate 控制保留多少上一隐状态

E. GRU 的 reset gate 决定遗忘多少上一隐状态

> **答案 A, C, D, E。**
> **陷阱**：B 的 **ReLU** 是错的（应为 sigmoid）——真题里你正是在这条上栽过。

预测 11（W4，概率高）

> 关于评估指标，哪些说法是**正确**的?

A. BLEU 主要基于 n-gram 的 precision

B. ROUGE 主要基于 n-gram 的 recall

C. F1 是 precision 与 recall 的调和平均

D. BLEU 主要基于 recall，ROUGE 主要基于 precision

E. METEOR 包含 chunk penalty

> **答案 A, B, C, E。**
> **陷阱**：**D 把 BLEU/ROUGE 的侧重对调**——这是 W4 真题的唯一扣分项，原样重现概率高。

预测 12（W4，概率中）

> 关于各分类器的原理，哪些说法是**正确**的?

A. Naïve Bayes 使用 Bag-of-Words，丢弃词的语法与结构

B. Naïve Bayes 会考虑词的语法结构来计算类别概率

C. SVM 通过寻找最大间隔超平面分隔数据

D. ELM 需要通过反向传播持续更新权重

E. 高斯过程把整个函数建模为随机变量，用 mean 和 covariance 刻画

> **答案 A, C, E。**
> **陷阱**：B 与 A 正好相反；D 的 ELM **不做 backprop**（权重只更新一次）。（B、D 都是真题原句改写。）

📙 填空预测（3 道）

> 题型说明：**每空只填一个词或一个字母，拼写必须正确，大小写不敏感。**

预测 13（W2，概率高）

> 在 spaCy 中，要获取每个命名实体的**文本**，应使用 `entity._____` 。

答案：`text`

若要获取实体**类型标签**，应使用 `entity._____` → **答案**：`label_`（**注意结尾下划线，但填空可能不考下划线；若题干强调"标签"则填** `label_`）

预测 14（W1，概率高）

把 `easily` 通过 `PorterStemmer` 处理后得到的结果是 `_____`。

答案：`easili`（小写 e-a-s-i-l-i）

这是本课程最经典的一个具体输出，值得直接背下来。

预测 15（W4，概率中）

衡量 machine translation 质量、侧重 **precision** 的指标是 `_____`；

衡量 summarization 质量、侧重 **recall** 的指标是 `_____`。

答案：`BLEU`（或 `bleu`）；`ROUGE`（或 `rouge`）

7.5 高概率考点 Top 10（考前最后 30 分钟看这张）

#	考点	一句话
1	**stemming vs lemmatization**	root form vs dictionary form；`easily → easili`
2	**spaCy 属性下划线**	`label_` / `pos_` / `dep_` / `orth_`，少一个下划线就错
3	**`get_feature_names_out()` 按字母序**	不是出现顺序
4	**`n_components`：int 是个数，float 是方差比例**	0.95 ≠ 95 个成分
5	**输出层与 loss 的任务匹配**	二分类 `Dense(1, sigmoid)`；多分类 `softmax` + `categorical_crossentropy`
6	**缩放 ≠ 归一化 ≠ 掩码**	√d_k 缩放；softmax 归一化；masked_fill 掩码
7	**Residual 不归一化；LayerNorm 才归一化**	residual 缓解 **vanishing**（不是 exploding）
8	**LSTM 门用 sigmoid；候选状态用 tanh**	forget gate 不是 ReLU
9	**BLEU = precision；ROUGE = recall**	别对调
10	**绝对化措辞要怀疑**	only / always / every / does not

7.6 如果只剩 1 小时

按这个顺序看，每分钟都有回报：

剩余时间	看什么	为什么
前 15 min	§ 7.5 高概率考点 Top 10	覆盖最高频的概念陷阱
中 20 min	§ 五 必背拼写表（API / 术语）	应对填空与代码题的拼写考点
中 15 min	§ 3 各周的"真题印证"框	直接看老师怎么设坑
后 10 min	§ 2.3 八大干扰项手法	上场前的"防骗清单"

附 A：真题全景表（去重后 40 题）

周	题号	主题	正确答案
W1	1	PorterStemmer 的作用	convert words to **root forms**
W1	2	`lemmatize('running', pos='v')`	find the **base form** of the verb
W1	3	sent→word→lemma→pos_tag pipeline	E（唯一三步嵌套正确）
W1	4	stopwords 过滤	**Removing stopwords**
W1	5	`re.sub` 去标点	`'[^\w\s]'`
W2	1	打印 NER 文本	`[entity.text for entity in doc.ents]`
W2	2	抽取含 ORG 的句子	`"ORG" in [ent.label_ for ent in sent.ents]`
W2	3	Lemmatize + unique	`set(...)` + `word_tokenize` + `WordNetLemmatizer()`
W2	4	WSD 算法名	**`lesk`**
W2	5	`re.sub('..g.*d', ...)` 输出	`Harper is the goodest girl.`（只替换 `"a good"`）
W2	6	Dictionary-based WSD 特征	不需 labeled data / 用 gloss / 测上下文重叠
W2	7	NER vs DP	都用 spaCy；NER 查答案、DP 标主宾语
W2	8	DP 的 INCORRECT 陈述	head/modifier 对调；"window size 4"
W2	9	POS 的 INCORRECT 描述	POS 定义被换成 dependency；"only way"
W2	10	NER 的正确描述	Google 搜索 / 信息抽取 / 答案检测 / 低阶检索
W3	1	`tagged_lemma` 对 TF-IDF 的好处	归并词形变体、**降低词表稀疏度**
W3	2	`TfidfVectorizer(stop_words='english')` 词表	`['cat','dog','mat','rug','sat']`（**字母序**）
W3	3	`PCA(n_components=0.95)`	保留**解释 95% 方差**的成分
W3	4	SRL：`Students learn NLP in NTU.`	Agent: Students / Patient: NLP / Predicate: learn
W3	5	TF-IDF 词表长度	**3**（apple / banana / orange）
W4	1	Topic modeling 算法描述	pLSA 概率混合 / LDA 无视上下文 / LSA 用 TF-IDF
W4	2	CRF 的 "DO NOT describe"	minimize log-lik / generative / only Quasi-Newton
W4	3	评估指标	METEOR chunk penalty / BLEU n-gram / F1 调和平均 / AUC-ROC tradeoff
W4	4	Word2vec / GloVe 函数	GloVe `f(x)` 抑制常见词对 / Word2vec Softmax 直觉
W4	5	分类器原理	SVM 超平面 / GP 随机函数
W5	1	`clip_grad_norm_`	prevent **exploding** gradients by capping the norm

周	题号	主题	正确答案
W5	2	PyG MessagePassing	adjacency matrix × node features → 传给邻居
W5	3	二分类输出层	`Dense(1, activation='sigmoid')`
W5	4	多分类 compile	`adam` + `categorical_crossentropy` + `accuracy`
W5	5	`RMSprop(lr=0.01)`	determines the **speed of convergence**
W6	1	Regularisation 的 INCORRECT	"increases variance and bias"；"weight decay = 丢神经元"
W6	2	Transformer 组件	masked MHA 定义；parallel heads
W6	3	ViT	CLIP/DALL・E；远距离交互；patch as token
W6	4	LSTM/GRU 门	output gate / update gate / reset gate
W6	5	Encoder-Decoder	自回归；只看已生成；encoder 吃 embedding
W7	1	softmax 的作用	转成**概率分布**
W7	2	self_attn vs cross_attn	target 内部 vs target-source
W7	3	ToT vs GoT	GoT：thoughts 是顶点、edges 表依赖
W7	4	除以 √d_k	保持**可比尺度**，防数值过大
W7	5	`nn.LayerNorm`	沿 **feature 维度**逐 token 归一化

附 B：材料与边界说明

本册依据

来源	用途
`mid-term-exam/requirement.txt`	考试规则（§一）
`mid-term-exam/week1–week7/*.png`	50 张周测回顾截图 → 去重后 **40 题**（§二、§三、附 A）
`Week{1,2,3,…,10}MCQ.md`	**练习题**文本，作为同周考点的补充（已与真题分开标注）
`week*/Week *.ipynb`	代码出处
`Learning_note.md`、`week5/Week5_Notes.md`	知识点与课程安排

已知边界

- **截图去重**：`week1/6–10.png` 与 `1–5.png` **完全重复**。`week2/` 的 10 张是**两组不同的 5 题**（第一组单选、第二组多选），不作重复处理。
- ⚠️ **真题 ≠ 练习题**：`week3/` 与 `week4/` 的归档真题与仓库内 `Week3MCQ.md` / `Week4MCQ.md` 的**题目内容不一致**（真题更偏"代码默认行为 + 概念多选"，练习题偏"missing code + 参数名"）。本册以**真题为主**，练习题作为同周考点补充单独列在每周末尾。

- **判分百分比**：截图中的 `0%` / `−20%` / `33.33%` 等是**回顾页显示的得分权重**，不代表选项被选择的分布，考试时不可见。
- **范围歧义**：官方 lesson plan 写 W1–6，归档含 W7。本册按 W1–7 覆盖。
- **GNN 内容**：W5 的 `MessagePassing` 在仓库课件/笔记中无出处，来自课堂短讲，依据截图 Feedback 整理。
- **答案归属**：所有"正确答案"均取自截图回顾页的 `Correct answer` **标记**与 Feedback 原文。带 `Incorrect:` 前缀 + 粉红高亮的行是**学生自己的错选**，一律未参与答案判定。
- **周测 vs 期中规则差异**：§〇 的期中规则取自 `requirement.txt`；§二 的判分机制描述的是**周测**（回顾页截图），两者不同，已分别标注。

OCR 复核记录（第二轮全量核对后）

本册答案键经**两轮独立抽取**（每轮 5 个并行子代理逐张读图）。40 题的正确选项**两轮结果完全一致**，且与每张图右上角得分的算术自洽（如 `week6/1`：+5(B) −2(C) −2(E) = 1/10 ✓）。

复核中修正的**两处代码文本**（**答案本身无误**，是代码抄录精度问题）：

位置	原抄录	复核后	影响
`week2/5.png` Q5 正则	`'.*g.*d'`	`'..g.*d'`（两个点，`d` 后无星号）	原抄录实际跑出的结果**不是任何选项**；修正后精确得到选项 A。已在正文加逐字符拆解
`week1/5.png` Q5 选项 A	`'[\.,;!?\"']'`	`'[\.,;:!?\']'`（含冒号）	该项是 0% 的 near miss，不影响答案

> 教训：正则类题目**必须逐字符核对**——`.*` 与 `..`、`d` 后有无 `*`，跑出来的结果完全不同。